

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 2 – Seminar on Emerging Technologies

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO5	Assessment:	Assessment Tool 2 – Seminar on Emerging Technologies
Weightage:	20%	Total Marks:	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	Chodam Nisha	Reg. No.:	192324306
Date:	25-08-2026	Duration:	60 minutes

Code of Conduct

I Ch.Nisha(192324306) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ch.Nisha

1. Introduction to the Emerging Technology

Techniques for Minimizing Resource Usage are methods used to reduce the amount of **CPU, memory, storage, network bandwidth, battery power, and other computing resources** required by a software system.

- Modern applications process large amounts of data.
- Excessive resource usage can make applications slow and expensive.
- Efficient software uses only the resources that are actually required.
- These techniques improve performance, scalability, reliability, and energy efficiency.
- They are especially important in cloud computing, mobile applications, IoT, AI systems, and data centers.

Simple example:

Instead of loading 1,000 images at once, an application loads only the images currently visible on the screen. This reduces memory and network usage.

2. Need and Motivation

Resource minimization is needed because computing resources are limited.

Main reasons:

- Reduce memory consumption
- Reduce CPU usage
- Save battery power
- Reduce storage requirements
- Reduce network traffic
- Decrease cloud computing costs
- Improve application response time
- Support more users simultaneously
- Reduce energy consumption

Example:

A mobile application that continuously uses GPS, camera, and internet services will consume more battery. Optimizing when these services are used can significantly reduce resource consumption.

3. Working Principle / Architecture

The basic principle is:

Monitor → Identify → Optimize → Execute → Measure

Working:

1. **Monitor resources**
 - Measure CPU, memory, storage, network, etc.
2. **Identify unnecessary usage**
 - Find inefficient algorithms, unused objects, repeated database queries, etc.
3. **Apply optimization techniques**
 - Caching
 - Compression
 - Lazy loading
 - Efficient algorithms
 - Database optimization
4. **Execute optimized software**
 - The application performs the same task using fewer resources.
5. **Measure performance**

- Compare resource usage before and after optimization.
-

Simple Architecture

User Request → Application → Resource Monitoring → Optimization Techniques → Efficient Resource Usage → Output

4. Key Components and Technologies

Important techniques:

1. Efficient Algorithms

- Choose algorithms with lower time and space complexity.
- Example: Using binary search instead of linear search for sorted data.

2. Caching

- Frequently used data is temporarily stored.
- Avoids repeatedly retrieving the same data.

3. Lazy Loading

- Resources are loaded only when required.
- Commonly used for images, videos, and web pages.

4. Data Compression

- Reduces the size of files and network data.
- Examples: ZIP, GZIP, image compression.

5. Database Optimization

- Use indexing.
- Avoid unnecessary queries.
- Retrieve only required data.

6. Memory Management

- Remove unused objects and release memory.
- Prevents memory leaks.

7. Cloud Resource Optimization

- Dynamically increase or decrease computing resources based on demand.

8. Code Optimization

- Remove unnecessary calculations and repeated operations.

5. Applications / Real-World Use Cases

1. Mobile Applications

Resource optimization helps reduce battery and memory usage.

Example: Instagram/YouTube-style applications use image and video compression.

2. Cloud Computing

Cloud platforms allocate resources according to workload.

Example: When website traffic is low, fewer servers can be used.

3. IoT Devices

IoT devices often have limited memory, processing power, and battery.

Example: A smart sensor sends data only when a significant change occurs instead of continuously sending data.

4. Web Applications

Techniques such as caching, compression, and lazy loading reduce page loading time.

5. AI and Machine Learning

Model compression and quantization reduce the computational resources required by AI models.

6. Data Centers

Efficient resource allocation reduces electricity consumption and operational costs.

6. Advantages and Limitations

Advantages

- Reduces CPU usage
- Reduces memory consumption
- Saves battery power
- Reduces cloud costs
- Improves application speed
- Improves scalability
- Reduces network bandwidth
- Reduces energy consumption

- Provides better user experience

Limitations

- Optimization can increase development complexity.
 - Some optimization techniques require additional tools.
 - Excessive optimization can make code difficult to understand.
 - Caching requires additional storage.
 - Compression may require additional processing time.
 - Optimization may require continuous monitoring.
-

7. Recent Developments and Trends

Current resource-minimization approaches increasingly focus on **cloud-native systems, edge computing, AI optimization, and energy-efficient computing.**

Important trends:

- **Serverless computing** – resources are allocated only when functions execute.
- **Edge computing** – processing is moved closer to users/devices.
- **AI model compression** – reduces model size and computation.
- **Quantization** – uses lower-precision values to reduce memory and processing requirements.
- **Container optimization** – lightweight containers reduce resource overhead.
- **Green computing** – focuses on reducing energy consumption.
- **Automatic resource scaling** – resources are increased or decreased according to workload.
- **Energy-aware software** – applications are designed to consume less power.

8. Challenges and Future Scope

Challenges

- Finding the correct balance between performance and resource usage.
- Monitoring resources in real time.
- Handling unpredictable workloads.

- Maintaining software quality while optimizing.
- Managing distributed and cloud-based systems.
- Avoiding performance degradation due to excessive optimization.

Future Scope

- AI-based automatic optimization
- Self-optimizing software
- Energy-efficient AI systems
- Green cloud computing
- Intelligent resource allocation
- Advanced edge computing
- Low-power IoT systems
- Carbon-aware computing

In the future, software may automatically detect high resource consumption and optimize itself without requiring manual intervention.

9. Case Study / Example

Case Study: YouTube Video Streaming

A video streaming application can consume large amounts of **network bandwidth, CPU, memory, and battery**.

To minimize resource usage:

Without optimization:

- High-quality video is always loaded.
- Large amounts of data are downloaded.
- More battery and network resources are consumed.

With optimization:

- **Adaptive streaming** selects video quality based on network conditions.
- **Caching** stores frequently accessed content.
- **Compression** reduces video size.
- **Lazy loading** loads content when required.

- Efficient memory management prevents unnecessary data from remaining in memory.

Result:

Less bandwidth + Less memory + Less CPU + Less battery usage → Better performance

Conclusion

Techniques for minimizing resource usage are essential for developing efficient, fast, scalable, and sustainable software systems. By using efficient algorithms, caching, compression, lazy loading, database optimization, cloud scaling, and memory management, applications can perform the same tasks while consuming fewer resources.

These techniques are becoming increasingly important with the growth of cloud computing, IoT, AI, mobile applications, and green computing.

References

1. Ian Sommerville, *Software Engineering*, Pearson.
2. Abraham Silberschatz, Peter B. Galvin, Greg Gagne, *Operating System Concepts*.
3. Andrew S. Tanenbaum, *Modern Operating Systems*.
4. Thomas H. Cormen et al., *Introduction to Algorithms*.
5. Martin Kleppmann, *Designing Data-Intensive Applications*.
6. Research literature on Green Computing and Energy-Efficient Software Engineering.
7. Research literature on Cloud Resource Optimization and Dynamic Resource Allocation.



